Engenharia de Software

5/3/2026

TP2

Velocidade e Qualidade com Estruturas de Dados e Algoritmos

Professor(a): Ricardo Pires Mesquita

Samuel Hermany
INSTITUTO INFNET

LINK GITHUB

https://github.com/faculdade-infnet/VI-1-velocidade-qualidade-com-algoritmos/tree/main/TP2
Obs. No GitHub consta além de um arquivo com código de cada questão, tem um outro com todas as questões e ordem, acredito que seja mais fácil para corrigir. tp1-todos-exercícios

1. QUESTÃO

Aqui para encontrar o maior número único e garantir a complexidade **O(n)**, separei o projeto em 3 grupos, primeiro com laços aninhados, segundo com hash table e um terceiro para testes e comparação entre os dois métodos.
Explicações:

- Laços aninhados O(n²)
  - No 1 array é percorrido todos os itens que internamente tem um 2 array onde encontrando os itens duplicados e a cada item duplicado somo +1 ao contador do item.
  - Então após o loop for, no if é verificado se o número de vezes que o item se repete = 1 e no if seguinte eu verifico se o valor atual do loop externo é vazio ou se é maior que o maior valor atual e então eu substituo ele.
  - Por fim, após percorrer todos os elementos do array no loop retorna o maior valor único encontrado
- Hash Table O(n)
  - Criei uma classe padrão para inserir, buscar e calcular a posição de cada item na Hash Table.
  - Criei mais uma função onde:
    - Obtenho a tabela gerada pela função HashTable
    - No primeiro arrays, percorro para encontrar e somar número de ocorrências de cada item
    - No segundo array percorro cada tabela dentro do grupo de tabelas gerado pelo HashTable e percorro cada tabela obtendo chave e valor, onde verifico se o valor=1 (nº ocorrências = 1)
    - No último if verifico se o valor é **None** ou se o valor da chave atual é maior que o maior valor armazenado e então atualizo se a condição for verdadeira.
    - Por fim, fora dos dois “loop for” eu retorno o maior valor encontrado.

- Testes
  - Criei 5 listas aleatórias de 5 a 20 itens contendo números de 1 a 10
  - Percorro todas as 5 listas e para cada uma das listas executo a função de laços aninhados e HashTable, exibindo em tela o maior valor encontrado para cada uma das funções.

1 - Otimização de Código Quadrático com Hash Table

1.1- Function - O(n^2) Maior número com dois laços aninhados

[1]
✓ 0s

```python
# Encontrar o maior número único
def greatestNumber(array):
    comparacoes = 0
    maiorValorUnico = None

    for i in array:
        repeticoes = 0
        # Encontra dos valore duplicados comparando cada i com todos j
        for j in array:
            # Se encontrarmos outro valor maior que i, então, i não é o maior:
            comparacoes += 1
            if j == i:
                repeticoes += 1

        # Se aparece apenas uma vez
        if repeticoes == 1:
            if maiorValorUnico is None or i > maiorValorUnico:
                maiorValorUnico = i

    return maiorValorUnico, comparacoes
```

[3]
✓ 0s

```python
# Encontrar o maior número único com Hash Table --------
def greatestNumberHashTable(array):
    hash_table = HashTable(len(array))
    maiorValorUnico = None
    acessos = 0

    # Inserir todos itens, contanto frequencia (i= numero)
    for i in array:
        acessos += 1
        numero_encontrado = hash_table.buscar(i)            # busca o numero na hash table
        if numero_encontrado is None:
            acessos += 1
            hash_table.inserir(i, 1)                        # insere o numero se ele não foi encontrado
        else:
            acessos += 1
            hash_table.inserir(i, numero_encontrado + 1)    # atualiza o contador de ocorrencias do número

    print(hash_table.tabela)

    # Percorrer todos os buckets da hash_table
    for bucket in hash_table.tabela:
        acessos += 1
        for chave, valor in bucket:
            acessos += 1
            # se a contagem for =1, número único
            if valor == 1:
                if maiorValorUnico is None or chave > maiorValorUnico:
                    maiorValorUnico = chave

    return maiorValorUnico, acessos
```

[3]
✓ 0s

```python
# Encontrar o maior número único com Hash Table --------
def greatestNumberHashTable(array):
    hash_table = HashTable(len(array))
    maiorValorUnico = None
    acessos = 0

    # Inserir todos itens, contanto frequencia (i= numero)
    for i in array:
        acessos += 1
        numero_encontrado = hash_table.buscar(i)            # busca o numero na hash table
        if numero_encontrado is None:
            acessos += 1
            hash_table.inserir(i, 1)                        # insere o numero se ele não foi encontrado
        else:
            acessos += 1
            hash_table.inserir(i, numero_encontrado + 1)    # atualiza o contador de ocorrencias do número

    print(hash_table.tabela)

    # Percorrer todos os buckets da hash_table
    for bucket in hash_table.tabela:
        acessos += 1
        for chave, valor in bucket:
            acessos += 1
            # se a contagem for =1, número único
            if valor == 1:
                if maiorValorUnico is None or chave > maiorValorUnico:
                    maiorValorUnico = chave

    return maiorValorUnico, acessos
```

1.3 - Tests

[4]

```python
import random
# Testando

# -------- GERAR LISTAS ALEATÓRIAS 5 A 20 NÚMEROS DO Nº 1 AO 100 --------
listas = []

for _ in range(5):
    tamanho = random.randint(5, 20)                         # define o número de itens na lista 5 a 20 itens
    lista = [random.randint(1, 10) for _ in range(tamanho)]  # criar lista com números de 1 a 10 (podendo repetir)
    listas.append(lista)                                    # adiciona a lista a listas(com 5 listas)

# -------- PERCORRE LISTAS E IMPRIME O MAIOR NÚMERO --------
for lista in listas:
    maiorValorUnico, comparacoes = greatestNumber(lista)
    print(f"### Tamanho: {len(lista)} ###")
    print("----- LAÇOS ANINHADOS -----")
    print(f'Número comparacoes: {comparacoes}')
    print(f'O maior valor é {maiorValorUnico} na lista: {lista}')

    print("----- HASH TABLE -----")
    maiorValorUnico, acessos = greatestNumberHashTable(lista)
    print(f'Número de acessos: {acessos}')
    print(f'O maior valor é {maiorValorUnico} na lista: {lista}\n')
```

```
### Tamanho: 13 ###
----- LAÇOS ANINHADOS -----
Número comparacoes: 169
O maior valor é 8 na lista: [4, 7, 8, 6, 9, 6, 5, 6, 10, 9, 4, 10, 7]
----- HASH TABLE -----
[[], [], [], [], [(4, 2)], [(5, 1)], [(6, 3)], [(7, 2)], [(8, 1)], [(9, 2)], [(10, 2)], [], []]
Número de acessos: 46
O maior valor é 8 na lista: [4, 7, 8, 6, 9, 6, 5, 6, 10, 9, 4, 10, 7]

### Tamanho: 7 ###
----- LAÇOS ANINHADOS -----
Número comparacoes: 49
O maior valor é 8 na lista: [8, 7, 9, 3, 3, 4, 9]
----- HASH TABLE -----
[[(7, 1)], [(8, 1)], [(9, 2)], [(3, 2)], [(4, 1)], [], []]
Número de acessos: 26
O maior valor é 8 na lista: [8, 7, 9, 3, 3, 4, 9]
```

```
### Tamanho: 20 ###
----- LAÇOS ANINHADOS -----
Número comparacoes: 400
O maior valor é 4 na lista: [4, 7, 9, 10, 5, 8, 5, 8, 6, 6, 7, 3, 9, 6, 7, 6, 3, 10, 9, 3]
----- HASH TABLE -----
[[], [], [], [(3, 3)], [(4, 1)], [(5, 2)], [(6, 4)], [(7, 3)], [(8, 2)], [(9, 3)], [(10, 2)], [], [], [], [], [], [], [], [], []]
Número de acessos: 68
O maior valor é 4 na lista: [4, 7, 9, 10, 5, 8, 5, 8, 6, 6, 7, 3, 9, 6, 7, 6, 3, 10, 9, 3]

### Tamanho: 6 ###
----- LAÇOS ANINHADOS -----
Número comparacoes: 36
O maior valor é 10 na lista: [5, 3, 7, 10, 7, 1]
----- HASH TABLE -----
[[], [(7, 2), (1, 1)], [], [(3, 1)], [(10, 1)], [(5, 1)]]
Número de acessos: 23
O maior valor é 10 na lista: [5, 3, 7, 10, 7, 1]

### Tamanho: 11 ###
----- LAÇOS ANINHADOS -----
Número comparacoes: 121
O maior valor é 9 na lista: [7, 7, 10, 6, 5, 3, 3, 9, 10, 5, 1]
----- HASH TABLE -----
[[], [(1, 1)], [], [(3, 2)], [], [(5, 2)], [(6, 1)], [(7, 2)], [], [(9, 1)], [(10, 2)]]
Número de acessos: 40
O maior valor é 9 na lista: [7, 7, 10, 6, 5, 3, 3, 9, 10, 5, 1]
```

2. QUESTÃO

Aqui para encontrar os elementos em comum entre dois arrays (comum + HashTable) e garantir a complexidade **O(n)**, separei o projeto em 3 grupos, primeiro com a class HashTable + function para popular ela com um array enviado, segundo com a função que verifica elementos em comum entre dois arrays e um terceiro para testes e retorno de lista com elemento em comum.

Explicações:

- Hash Table O(n)
  - Criei uma classe padrão para inserir, buscar e calcular a posição de cada item na Hash Table.
  - Criei mais uma função onde:
    - Obtenho a tabela gerada pela função HashTable
    - No primeiro arrays, percorro para encontrar e somar número de ocorrências de cada item
    - No segundo array percorro cada tabela dentro do grupo de tabelas gerado pelo HashTable e percorro cada tabela obtendo chave e valor, onde verifico se o valor=1 (nº ocorrências = 1)
    - No último if verifico se o valor é None ou se o valor da chave atual é maior que o maior valor armazenado e então atualizo se a condição for verdadeira.
    - Por fim, fora dos dois “loop for” eu retorno o maior valor encontrado.
- Function Intersection → **encontra elementos em comum, ordena**
  - Populo o HashTable com o primeiro array
  - No loop for percorro todo o array2 e dentro dele busco o valor atual percorrido dentro do HashTable com array1
  - Verifico com if se o valor obtido é diferente de None e se valor >0 e então eu adiciono a lista de **elementos_comuns** e atualizo o HashTable para não buscar se houver outro valor igual
  - Então ordeno a lista (**elementos_comuns**) obtida e retorno ela.
- Testes → **validação prática com grandes volumes de dados**
  - Criei 5 listas aleatórias de 50 mil a 100 mil itens contendo números de 1 a mil

- Percorro todas as 5 listas e para cada uma das listas executo a função Intersection que retorna os itens em comum nas duas listas, exibindo em tela um array planificado(string) com os elementos em comum.

2 - Interseção de Arrays em Tempo Linear

2.1 - Class Hash Table

```python
class HashTable:
    # O(n) pois cria "n" listas vazias
    def __init__(self, tamanho):
        self.tamanho = tamanho
        self.tabela = [[] for _ in range(tamanho)]  # cria o numero(tamanho) de listas vazias solicitadas

    # transforma a chave em um índice válido do array (45 % 10 = 5 posição)
    def _funcao_hash(self, chave):
        return chave % self.tamanho

    # insere um novo valor caso ele não exista, se existir atualiza
    def inserir(self, chave, valor):
        # calcula o índice
        index= self._funcao_hash (chave)
        bucket = self.tabela [index]

        # verificar se a chave já existe e atualização do valor
        for i, (chave_existente, _) in enumerate (bucket):
            if chave_existente == chave:
                bucket[i] = (chave, valor)
                return
        # se a chave não existe, adicionar o novo par chave-valor no bucket
        bucket.append((chave, valor))

    # busca um valor dentro do bucket/tabela
    def buscar(self, chave):
        # calcula o índice
        index= self._funcao_hash (chave)
        bucket = self.tabela [index]
        # procurar pela chave e retornar o valor correspondente
        for chave_existente, valor in bucket:
            if chave_existente == chave:
                return valor

        return None # chave não encontrada!
```

```python
# Popular array na Hash Table
def populateHashTable(array):
    hash_table = HashTable(10)

    for item in array:
        if hash_table.buscar(item) is None:
            hash_table.inserir(item, 1)
        else:
            valor = hash_table.buscar(item)
            hash_table.inserir(item, valor + 1)

    return hash_table
```

2.2 - Function - Intersecção

```python
# Encontra os valores comuns entre os dosi arrays
def intersection(array1, array2):
    hash_table = populateHashTable(array1)    # primeiro array, pupular
    elementos_comuns = []                     # segundo array
    print(hash_table.tabela)

    for i in array2:
        valor = hash_table.buscar(i)
        if valor is not None and valor > 0:
            elementos_comuns.append(i)
            hash_table.inserir(i, 0)  # marca como já usado, evitando duplicata

    elementos_comuns.sort()
    return elementos_comuns
```

2.3 - Test

```python
import random
# Testando

# -------- GERAR LISTAS ALEATÓRIAS 5 A 20 NÚMEROS DO Nº 1 AO 100 --------
listas = []

for _ in range(6):
    tamanho = random.randint(50000, 100000)                  # define o número de itens na lista 5 a 20 itens
    lista = [random.randint(1, 1000) for _ in range(tamanho)]  # criar lista com números de 1 a 10 (podendo repetir)
    listas.append(lista)                                     # adiciona a lista a listas(com 5 listas)

# -------- PERCORRE LISTAS E IMPRIME O VALORES EM COMUM --------
for lista1, lista2 in zip(listas, listas[1:]):
    elementos_comuns = intersection(lista1, lista2)

    print(f'Lista 1: {lista1}')
    print(f'Lista 2: {lista2}')
    lista_planificada = ", ".join(str(x) for x in elementos_comuns)
    print(f'Elementos em comum: {lista_planificada}\n')
```

```
[[(560, 90), (680, 92), (170, 87), (150, 100), (180, 88), (330, 84), (930, 104), (820, 69), (590, 74), (860, 85), (440, 83), (600, 95), (60, 87), (520, 94), (410, 97), (500, 93), (920, 90), (10, 108), (850, 86), (750, 91), (
Lista 1: [762, 912, 560, 680, 170, 79, 699, 757, 915, 639, 346, 695, 329, 745, 385, 392, 507, 235, 63, 914, 150, 854, 792, 824, 25, 149, 507, 272, 968, 215, 981, 657, 4, 74, 614, 605, 180, 428, 618, 104, 894, 835, 482, 284,
Lista 2: [145, 387, 831, 714, 227, 3, 748, 461, 362, 68, 725, 850, 555, 487, 334, 688, 79, 603, 42, 247, 555, 485, 250, 818, 613, 823, 344, 831, 516, 562, 847, 672, 632, 789, 479, 676, 880, 830, 115, 954, 971, 186, 227, 88,
Elementos em comum: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 5

[[(850, 58), (250, 69), (880, 49), (830, 56), (750, 69), (480, 70), (790, 56), (450, 60), (660, 65), (330, 56), (680, 56), (810, 72), (520, 63), (290, 66), (90, 64), (180, 50), (630, 58), (170, 66), (780, 62), (190, 65), (57
Lista 1: [145, 387, 831, 714, 227, 3, 748, 461, 362, 68, 725, 850, 555, 487, 334, 688, 79, 603, 42, 247, 555, 485, 250, 818, 613, 823, 344, 831, 516, 562, 847, 672, 632, 789, 479, 676, 880, 830, 115, 954, 971, 186, 227, 88,
Lista 2: [353, 685, 551, 136, 134, 630, 675, 410, 928, 728, 959, 745, 862, 355, 671, 353, 819, 363, 606, 717, 985, 785, 260, 362, 552, 27, 292, 199, 688, 182, 368, 165, 864, 523, 586, 610, 388, 781, 54, 319, 655, 428, 214, 9
Elementos em comum: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 5

[[(630, 75), (410, 63), (260, 76), (610, 88), (920, 58), (510, 58), (940, 70), (480, 46), (70, 61), (300, 67), (490, 55), (880, 65), (190, 72), (710, 70), (580, 59), (1000, 85), (780, 84), (310, 84), (200, 65), (340, 77), (2
Lista 1: [353, 685, 551, 136, 134, 630, 675, 410, 928, 728, 959, 745, 862, 355, 671, 353, 819, 363, 606, 717, 985, 785, 260, 362, 552, 27, 292, 199, 688, 182, 368, 165, 864, 523, 586, 610, 388, 781, 54, 319, 655, 428, 214, 9
Lista 2: [175, 786, 912, 60, 156, 984, 29, 145, 230, 717, 879, 304, 229, 399, 257, 593, 308, 341, 220, 255, 736, 907, 968, 188, 266, 3, 455, 456, 859, 494, 586, 820, 970, 642, 909, 365, 596, 548, 347, 177, 237, 561, 589, 419
Elementos em comum: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 5

[[(60, 57), (230, 54), (220, 64), (820, 44), (970, 59), (100, 48), (500, 50), (920, 53), (1000, 58), (950, 59), (770, 51), (260, 60), (20, 61), (680, 64), (40, 56), (400, 47), (590, 64), (50, 57), (140, 65), (880, 39), (610,
Lista 1: [175, 786, 912, 60, 156, 984, 29, 145, 230, 717, 879, 304, 229, 399, 257, 593, 308, 341, 220, 255, 736, 907, 968, 188, 266, 3, 455, 456, 859, 494, 586, 820, 970, 642, 909, 365, 596, 548, 347, 177, 237, 561, 589, 419
Lista 2: [878, 424, 988, 42, 858, 877, 242, 909, 44, 591, 192, 396, 75, 684, 481, 992, 484, 856, 279, 566, 686, 560, 726, 989, 515, 237, 17, 517, 699, 375, 412, 25, 408, 725, 817, 762, 343, 561, 448, 606, 451, 992, 14, 991,
Elementos em comum: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 5

[[(560, 51), (720, 50), (250, 59), (510, 45), (750, 78), (170, 42), (620, 58), (860, 55), (680, 60), (670, 52), (780, 66), (120, 54), (890, 51), (650, 50), (410, 53), (690, 61), (370, 53), (440, 49), (660, 50), (700, 56), (4
Lista 1: [878, 424, 988, 42, 858, 877, 242, 909, 44, 591, 192, 396, 75, 684, 481, 992, 484, 856, 279, 566, 686, 560, 726, 989, 515, 237, 17, 517, 699, 375, 412, 25, 408, 725, 817, 762, 343, 561, 448, 606, 451, 992, 14, 991,
Lista 2: [98, 915, 360, 234, 205, 216, 432, 756, 449, 720, 929, 747, 981, 347, 73, 610, 218, 89, 293, 343, 263, 440, 983, 187, 804, 611, 57, 33, 167, 624, 907, 399, 626, 110, 124, 252, 913, 659, 44, 208, 975, 728, 977, 340,
Elementos em comum: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 5
```

3. QUESTÃO

Aqui para encontrar o primeiro valor duplicado e garantir a complexidade **O(n)**, separei o projeto em 3 grupos, primeiro com a class HashTable + function para gerar palavras aleatórias , segundo com a função que verifica o primeiro elemento duplicado e um terceiro para testes e impressão do item encontrado na segunda função.

Explicações:

- Hash Table O(n)
  - Criei uma classe padrão para inserir, buscar e calcular a posição de cada item na Hash Table
  - Criei mais uma função onde:
    - Obtenho a tabela gerada pela função HashTable
    - No primeiro arrays, percorro para encontrar e somar número de ocorrências de cada item
    - No segundo array percorro cada tabela dentro do grupo de tabelas gerado pelo HashTable e percorro cada tabela obtendo chave e valor, onde verifico se o valor=1 (nº ocorrências = 1)
    - No último if verifico se o valor é None ou se o valor da chave atual é maior que o maior valor armazenado e então atualizo se a condição for verdadeira.
    - Por fim, fora dos dois “loop for” eu retorno o maior valor encontrado.
- Function **firstDuplicatedValue → encontra a primeira string duplicada**
  - Populo o HashTable vazio do tamanho do meu array
  - No loop for percorro todo o array e dentro dele busco o valor atual percorrido dentro do HashTable
  - Verifico se o valor obtido é diferente de **None**, se verdadeiro, então eu já encontrei meu valor duplicado
  - Senão adiciona na HashTable o valor atual de leitura
  - Por fim retorno o item duplicado que foi encontrado.
- Testes → **validação prática com grandes volumes de dados**
  - Primeiro loop for: percorro 5 vezes, gerando 5 listas e para cada uma das listas eu:
    - É definido um tamanho aleatório de 5 a 15 palavras
    - Um lista com palavras aleatórias de 3 a 10 caracteres
    - Crio um índice aleatório
    - Adiciono um novo item na lista = ao que possui o mesmo índice aleatório gerado
    - Adiciono a lista ao array de listas
  - Percorro todas as 5 listas e para cada uma das listas executo a função **firstDuplicatedValue** que retorna os primeiro item duplicado e exibo em tel o valor encontrado.

3 - Primeiro valor duplicado | apenas 1 par duplicado | Hash Table

3.1 - Class Hash Table

```python
class HashTable:
    # O(n) pois cria "n" listas vazias
    def __init__(self, tamanho):
        self.tamanho = tamanho
        self.tabela = [[] for _ in range(tamanho)]  # cria o numero(tamanho) de listas vazias solicitadas

    # transforma a string em uma chave de índice válido somando códigos ASCII
    def _funcao_hash(self, chave):
        # direita <--- esquerda - se for string/ percorre todos os caracteres/ obtém o número ASCII e soma os numeros
        chave_gerada = sum(ord(c) for c in str(chave))
        return chave_gerada % self.tamanho

    # insere um novo valor caso ele não exista, se existir atualiza
    def inserir(self, chave, valor):
        # calcula o índice
        index= self._funcao_hash (chave)
        bucket = self.tabela [index]

        # verificar se a chave já existe e atualização do valor
        for i, (chave_existente, _) in enumerate (bucket):
            if chave_existente == chave:
                bucket[i] = (chave, valor)
                return
        # se a chave não existe, adicionar o novo par chave-valor no bucket
        bucket.append((chave, valor))

    # busca um valor dentro do bucket/tabela
    def buscar(self, chave):
        # calcula o índice
        index= self._funcao_hash (chave)
        bucket = self.tabela [index]
        # procurar pela chave e retornar o valor correspondente
        for chave_existente, valor in bucket:
            if chave_existente == chave:
                return valor

        return None # chave não encontrada!
```

```python
import random
import string

# Gera palavras aleatórias
def gerar_palavra(tamanho=5):
    letras = string.ascii_lowercase
    return ''.join(random.choice(letras) for _ in range(tamanho))
```

3.2 - Function - First Duplicated Value

```python
# Retorne a primeira string duplicada
def firstDuplicatedValue (array):
    hash_table = HashTable(len(array))   # Cria um array vazio
    item_duplicado = None

    #print(hash_table.tabela)
    for item in array:
        valor = hash_table.buscar(item)
        if valor is not None:
            item_duplicado = item
            break
        else:
            hash_table.inserir(item, 1)   # marca como visto

    return item_duplicado
```

3.3 - Test

```python
listas = []

# listas aleatórias de string (start, stop, step)
for n in range(5):
    tamanho_lista = random.randint(5, 15)  # número de palavras na lista (5 a 15)
    lista = [gerar_palavra(random.randint(3, 10)) for _ in range(tamanho_lista)]   # gera uma lista de palavra podendo ter de 3 a 10 caracreres por sorteio
    # lista.append(lista[0])
    indice_aleatorio = random.randint(0, len(lista) - 1)   # define um indice aleatorio denttro do total de indices
    lista.append(lista[indice_aleatorio])                  # adicona a duplicata na lista
    listas.append(lista)

for lista in listas:
    item_duplicado = firstDuplicatedValue(lista)
    print(f"Lista utilizada: {lista}")
    print(f'Primeiro item duplicado : {item_duplicado}\n')
```

```
Lista utilizada: ['tuxrgx', 'mswvbcnvew', 'xzcwwihs', 'wzwpvz', 'vofprokjug', 'vdvotukspa', 'vofprokjug']
Primeiro item duplicado : vofprokjug

Lista utilizada: ['qfxbrvndrk', 'ahvrtwp', 'pcxlx', 'hbnfn', 'dlc', 'jegjjanh', 'hnqorteo', 'faxkadtri', 'wnxofol', 'faxkadtri']
Primeiro item duplicado : faxkadtri

Lista utilizada: ['sckitdnww', 'crbyker', 'frlcjkpb', 'ltzzqcu', 'pxrxewgnnp', 'nzel', 'uwdjyo', 'xqt', 'qskhnh', 'zbogox', 'bbvmokg', 'ruspt', 'jmwmb', 'snpgk', 'wpspdevjb', 'bbvmokg']
Primeiro item duplicado : bbvmokg

Lista utilizada: ['xdnvgosrn', 'jpvipicb', 'cirzshz', 'qjvqjnx', 'txe', 'vbpvdzi', 'ammg', 'zcug', 'zcug']
Primeiro item duplicado : zcug

Lista utilizada: ['cukruvwcw', 'vdzetnzp', 'dxsfrxxvpw', 'aarx', 'outw', 'outw']
Primeiro item duplicado : outw
```

4. QUESTÃO

Complexidade:**O(n)**, pois o algoritmo percorre cada letra do texto apensa uma vez.
Aqui busco identificar a letra ausente do alfabeto em uma string previamente informada.
Explicação:

- Criei uma função para verificar o que foi pedido acima.
- Criei uma array alfabeto com sub array **[letra, encontrada(True/False)]**, onde cada letra tem o valor default False e após ser encontrada seu valor é alterado para True.
- No “loop for” seguinte percorro a string informada, caractere por caractere, e então verifico se é uma letra de **a até z**, se a condição for verdadeira, então altero o valor default do sub array para True **[letra, True]**.
- Percorro o segundo “loop for” para criar dois arrays **[letras_encontradas]** e **[letras_nao_encontradas]** verificando se a posição 2 no sub array é True ou False.
- Retorno os dois arrays resultantes e planifico eles, separados os valores por **“,”**.
- Por fim, em tela são exibidas as duas strings com as letras encontradas e as não encontradas.

4 - Letra ausente no alfabeto

[1]
0s

```python
import string

def missing_letter(texto):
    # cria o alfabeto de "a" até "z", onde por default tudo é False
    alfabeto = [[chr(i), False] for i in range(ord('a'), ord('z') + 1)]

    # i = letra
    for i in texto:
        if 'a' <= i <= 'z':                # garante letras de 'a' até 'z'
            indice = ord(i) - ord('a')     # localiza o índice da letra no alfabeto
            # print(indice, "-", i)
            alfabeto[indice][1] = True     # Marca que localizou a letra no alfabeto

    letras_encontradas = []
    letras_nao_encontradas = []
    for i in alfabeto:
        # verifica se a segunda do sub array no alfabeto = False (letra não foi encontrada)
        if i[1] == False:
            # adiciona letra não econtrada na nova lista
            letras_nao_encontradas.append(i[0])
        else:
            letras_encontradas.append(i[0])

    return(letras_encontradas, letras_nao_encontradas)

# Testando
#texto = "hoje é um dia de sol que Parece não acabar mais"
texto = "acdefghijklmnopqrstuvyxZw"
#texto = "a D , . #"
(letras_encontradas, letras_nao_encontradas) = missing_letter(texto)
result_encontradas = ", ".join(letras_encontradas)
result_nao_encontradas = ", ".join(letras_nao_encontradas)
print(f"Letras faltando: ", result_nao_encontradas)
print(f"Letras encontradas: ", result_encontradas)
```

```
Letras faltando:  b, z
Letras encontradas:  a, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y
```

5. Questão

Aqui busco retornar o primeiro caractere não repetido de uma string previamente informada.
Explicação:

- Criei uma função para verificar o que foi pedido acima.
- Criei um dicionário **[caracteres_contagem]** para armazenar os caracteres encontrados e o número de ocorrências **{caractere : nº ocorrência}**.
- No “loop for” seguinte percorro a string informada, caractere por caractere, e então eu verifico se ela já está contida no dicionário, se a condição for verdadeira, então altero o atribui +1 ao valor atual de contagem, senão, eu atribuo o novo caractere ao dicionário com valor inicial de 1.
- Percorro o segundo “loop for” dentro do dicionário e verifico o primeiro caractere único encontrado, armazenando ele e finalizando o loop.
- Retorno os dois itens (variável, dicionário).
- Por fim, em tela são exibidos os dois itens acima, com o primeiro caractere encontrado e o dicionário com a contagem de caracteres resultante.

5 - Primeiro caractere não repetido

[1]
✓ 0s

```python
# contar frequencia em array com sub array = [[a, 1,], [b, 2], [c, ] ]
# para encontrar o primei caractere não repetido, após o for acim, percorrer a lista e enocntrar onde a segunda posição[1] = [caractere, 1]
import string

def first_letter(texto):
    # conta as ocorrências das letras encontradas {caractetere, contador}
    caracteres_contagem = {}

    # i = letra
    for i in texto:
        if i in caracteres_contagem:
            caracteres_contagem[i] += 1
        else:
            caracteres_contagem[i] = 1

    # armazena o primeiro caractere não repetido
    primeiro_caractere = "";
    for letra, i in caracteres_contagem.items():
        # verifica o primeiro valor = 1 pra o sub array no alfabeto (primeira letra encontrada)
        if i == 1:
            # atribui letra econtrada
            primeiro_caractere = letra
            break

    return(primeiro_caractere, caracteres_contagem)


texto1 = "aB3! xYz9@#LmN0$pQ7kL& tR2*e"
texto2 = "W8+vZ1^mN4% pQ6!sT9@uV2#Xy1$"
texto3 = "zA3&bC5*dE7!9fG@ hJ2#kL4%mN"
texto4 = "6&Op8( qR0)St2=Uv4?Wx6/Yz8"
texto5 = "Ab1!Cd3@Ef5#Gh7$Ij9%Kl0^Mn"
lista = [texto1, texto2, texto3, texto4, texto5]

for i, texto in enumerate(lista):
    i += 1
    (primeiro_caractere, caracteres_contagem) = first_letter(texto)
    print(f"Primeiro caractere não duplicado L{i}:", primeiro_caractere)
    print(f"Contagem de caracteres L{i}:\n", caracteres_contagem)
    print()
```

```
Primeiro caractere não duplicado L1: a
```

```
Primeiro caractere não duplicado L1: a
Contagem de caracteres L1:
 {'a': 1, 'B': 1, '3': 1, '!': 1, ' ': 2, 'x': 1, 'Y': 1, 'z': 1, '9': 1, '@': 1, '#': 1, 'L': 2, 'm': 1, 'N': 1, '0': 1, '$': 1, 'p': 1, 'Q': 1, '7': 1, 'k': 1, '&': 1, 't': 1, 'R': 1, '2': 1, '*': 1, 'e': 1}

Primeiro caractere não duplicado L2: W
Contagem de caracteres L2:
 {'W': 1, '8': 1, '+': 1, 'v': 1, 'Z': 1, '1': 2, '^': 1, 'm': 1, 'N': 1, '4': 1, '%': 1, ' ': 1, 'p': 1, 'Q': 1, '6': 1, '!': 1, 's': 1, 'T': 1, '9': 1, '@': 1, 'u': 1, 'V': 1, '2': 1, '#': 1, 'X': 1, 'y': 1, '$': 1}

Primeiro caractere não duplicado L3: z
Contagem de caracteres L3:
 {'z': 1, 'A': 1, '3': 1, '&': 1, 'b': 1, 'C': 1, '5': 1, '*': 1, 'd': 1, 'E': 1, '7': 1, '!': 1, '9': 1, 'f': 1, 'G': 1, '@': 1, ' ': 1, 'h': 1, 'J': 1, '2': 1, '#': 1, 'k': 1, 'L': 1, '4': 1, '%': 1, 'm': 1, 'N': 1}

Primeiro caractere não duplicado L4: &
Contagem de caracteres L4:
 {'6': 2, '&': 1, 'O': 1, 'p': 1, '8': 2, '(': 1, ' ': 1, 'q': 1, 'R': 1, '0': 1, ')': 1, 'S': 1, 't': 1, '2': 1, '=': 1, 'U': 1, 'v': 1, '4': 1, '?': 1, 'W': 1, 'x': 1, '/': 1, 'Y': 1, 'z': 1}

Primeiro caractere não duplicado L5: A
Contagem de caracteres L5:
 {'A': 1, 'b': 1, '1': 1, '!': 1, 'C': 1, 'd': 1, '3': 1, '@': 1, 'E': 1, 'f': 1, '5': 1, '#': 1, 'G': 1, 'h': 1, '7': 1, '$': 1, 'I': 1, 'j': 1, '9': 1, '%': 1, 'K': 1, 'l': 1, '0': 1, '^': 1, 'M': 1, 'n': 1}
```

6. Questão

Complexidade:**O(n²)**, pois o algoritmo como loop externo percorre a lista **n** vezes e no loop interno mais **n** vezes os elementos restantes, onde isso dá aproximadamente **n * n = O(n²)**.
O número de comparações tem a tendência de aumentar conforme **n** cresce, pois ele independe da ordem dos dados de entrada.
Já o número de trocas é menor quanto mais a lista ordenada estiver, havendo assim menos esforço computacional.
As variações de tempos são muito pequenas.

1 - Selection Sort

```python
import time

def selection_sort(lista):
    n = len(lista)
    comparacoes = 0 # número de comparações executadas
    trocas = 0      # número de trocas executadas

    # percorre a lista para localizar o menor elemento
    for i in range(n):
        min_index = i   # assume que o elemento atual é o menor

        # encontramos o menor elemento na parte não ordenada da lista
        for j in range(i + 1, n):   # Start index, Stop index
            comparacoes += 1
            # Se estiver fora de ordem → troca.
            if lista[j] < lista[min_index]:
                min_index = j   # Atualiza índice do menor elemento

        # trocando o menor elemento com o primeiro elemento da parte não ordenada
        if min_index != i:
            trocas += 1
            lista[i], lista[min_index] = lista[min_index], lista[i]

    print(f'\tNúmero comparacoes: {comparacoes}')
    print(f'\tNúmero trocas: {trocas}')

# testando
listas = [
    ("lista aleatoria", [47, 12, 89, 3, 76, 54, 28, 91, 65, 10, 38, 72, 5, 99, 41, 23, 84, 16, 58, 67, 2, 95, 31, 44, 79, 18, 60, 7, 88, 52]),
    ("lista ordenada", list(range(1, 60, 2))),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista pequena", [21, 28, 16, 24, 6, 3, 27, 22, 29, 1]),
    ("lista grande ordenada", list(range(60, 0, -2)))
]

for i, (nome, lista) in enumerate(listas, start=1):
    print(f"{i} - {nome}:")
    inicio_t = time.perf_counter()
    selection_sort(lista)
    total_linear = time.perf_counter() - inicio_t
    print(f"\tTempo: {total_linear:.5f} segundos = {lista}")
```

```
1 - lista aleatoria:
```

```
1 - lista aleatoria:
        Número comparacoes: 435
        Número trocas: 29
        Tempo: 0.00010 segundos = [2, 3, 5, 7, 10, 12, 16, 18, 23, 28, 31, 38, 41, 44, 47, 52, 54, 58, 60, 65, 67, 72, 76, 79, 84, 88, 89, 91, 95, 99]
2 - lista ordenada:
        Número comparacoes: 435
        Número trocas: 0
        Tempo: 0.00007 segundos = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59]
3 - lista invertida:
        Número comparacoes: 435
        Número trocas: 15
        Tempo: 0.00006 segundos = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60]
4 - lista pequena:
        Número comparacoes: 45
        Número trocas: 9
        Tempo: 0.00002 segundos = [1, 3, 6, 16, 21, 22, 24, 27, 28, 29]
5 - lista grande ordenada:
        Número comparacoes: 435
        Número trocas: 15
        Tempo: 0.00006 segundos = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60]
```

7. QUESTÃO

Complexidade

Melhor caso: O(n) → lista quase ordenada, comparações mínimas, pois elementos já estão quase na posição correta.

Pior caso: O(n²) → lista inversamente ordenada, muitas comparações e deslocamentos, pois cada elemento precisa ser comparado e deslocado até a posição correta.

O número de comparações e de deslocamentos é menor quando a lista é ordenada ou quase ordenada, pois nos dois casos a lista já está quase no formato esperado, havendo assim menos esforço computacional.

Para n=30 a diferença de tempo foi pouca, mas a tendência é aumentar conforme **n** cresce.

7 - Insertion Sort

```python
import time

def insertion_sort(lista):
    n = len(lista)
    comparacoes = 0 # número de comparações executadas
    deslocamentos = 0      # número de deslocamentos executadas

    # percorre a lista a partir do segundo elemento
    # Start index, Stop index
    for i in range(1, n):
        # o elemento atual é o escolhido para inserção
        chave = lista[i]
        # inicia a comparação a partir do elemento anterior da lista
        j = i - 1

        # mover para direita os elementos da sub-lista ordenada que são maiores do que "chave"
        while j >= 0:
            comparacoes += 1
            if lista[j] > chave:
                lista[j + 1] = lista[j]
                deslocamentos += 1
                j -=1
            else:
                break

        # insere chave na posição correta
        lista[j + 1] = chave

    print(f'\tNúmero comparacoes: {comparacoes}')
    print(f'\tNúmero deslocamentos: {deslocamentos}')


# testando
listas = [
    ("lista quase ordenada", [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 25, 24, 26, 27, 28, 29, 30]),
    ("lista quase ordenada", [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 16, 18, 19, 20, 21, 22, 23, 24, 26, 25, 27, 28, 29, 30, 31]),
    ("lista quase ordenada", [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 15, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 27, 29, 30, 31, 32, 33, 34]),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista invertida", list(range(80, 0, -3)))
]
```

```python
# testando
listas = [
    ("lista quase ordenada", [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 25, 24, 26, 27, 28, 29, 30]),
    ("lista quase ordenada", [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 16, 18, 19, 20, 21, 22, 23, 24, 26, 25, 27, 28, 29, 30, 31]),
    ("lista quase ordenada", [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 15, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 27, 29, 30, 31, 32, 33, 34]),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista invertida", list(range(80, 0, -3)))
]

for i, (nome, lista) in enumerate(listas, start=1):
    print(f"{i} - {nome}:")
    inicio_t = time.perf_counter()
    insertion_sort(lista)
    total_linear = time.perf_counter() - inicio_t
    print(f"\tTempo: {total_linear:.5f} segundos = {lista}")
```

```
1 - lista quase ordenada:
        Número comparacoes: 31
        Número deslocamentos: 2
        Tempo: 0.00003 segundos = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]
2 - lista quase ordenada:
        Número comparacoes: 31
        Número deslocamentos: 2
        Tempo: 0.00001 segundos = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31]
3 - lista quase ordenada:
        Número comparacoes: 31
        Número deslocamentos: 2
        Tempo: 0.00001 segundos = [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34]
4 - lista invertida:
        Número comparacoes: 435
        Número deslocamentos: 435
        Tempo: 0.00005 segundos = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60]
5 - lista invertida:
        Número comparacoes: 351
        Número deslocamentos: 351
        Tempo: 0.00004 segundos = [2, 5, 8, 11, 14, 17, 20, 23, 26, 29, 32, 35, 38, 41, 44, 47, 50, 53, 56, 59, 62, 65, 68, 71, 74, 77, 80]
```

8. QUESTÃO

FILA - pois o sistema atende as solicitações estritamente na ordem de chegada. A estrutura de dados que atende o que foi solicitado dentre as duas possíveis é fila, pois segue o princípio FIFO (First In, First Out), onde a primeira solicitação que entra é a primeira a ser atendida.
Operações envolvidas:

- Criar fila vazia
- Enfileirar - insere elemento no final da fila
- Desenfileirar - remove o primeiro item no inicio da fila
- Consultar o primeiro item
- Esvaziar fila, remove todos os itens
- Verificar se a fila está cheia
- Verificar se a fila está vazia

9. QUESTÃO

Neste exercício, construí uma pilha com capacidade fixa, com tratamento de erros, incluindo overflow (inserção em pilha cheia) e underflow (remoção em pilha vazia), sendo que para isso irei demonstrar o funcionamento com uma sequência de operações.
Explicação (os detalhes linha a linha estão comentados no código):

- Criei uma classe no primeiro bloco de código com as seguintes funções:
  - **init** - Iniciar/Criar nova pilha
  - **vazia** - verifica se pilha está vazia
  - **empilhar/push (adicionar)** - adiciona um novo item no topo da pilha
  - **desempilhar** - remove o item do top e retorna o item removido
  - **topo** - retorna o item do topo da pilha
  - **tamanho** - retorna o número de elementos da pilha
  - **esvaziar** - esvaziar pilha, remove todos os dados
  - **imprimir** - imprime todos os itens existentes na pilha
- Depois criei uma segundo bloco de código código para demonstrar o que foi pedido:
  - imprimir - exibe os dados iniciais na pilha a cada adição
  - 1 - topo - exibe o item no topo da pilha
  - 2 - tamanho - exibe o tamanho da pilha
  - 3 - desempilhar - remove um item no topo da pilha
  - imprimir - exibe os dados atuais na pilha
  - 4 - empilhar - adiciona um item no topo da pilha
  - imprimir - exibe os dados atuais na pilha
  - 5 - topo - exibe o item no topo da pilha
  - 6 - tamanho - exibe o tamanho da pilha
  - 7 - empilhar - adiciona um item no topo da pilha
  - imprimir - exibe os dados atuais na pilha
  - 8 - topo - exibe o item no topo da pilha
  - 9 - tamanho - exibe o tamanho da pilha
  - esvaziar - limpar os dados da pilha
  - 10 - tamanho - exibe o tamanho da pilha
  - desempilhar - remove um item no topo da pilha
  - imprimir - exibe os dados atuais na pilha

9 - Pilha com Controle de Erros

```python
class Pilha:
    # cria uma nova pilha vazia com capacidade máxima
    def __init__(self, capacidade):
        self.capacidade = capacidade        # adicionar atributo ao objeto
        self.itens = []

    # verifica se a pilha está vazia
    def vazia(self):
        return not self.itens

    # adiciona um novo item no topo da pilha (push)
    def empilhar (self, item):
        # Se pilha = capacidade máxima de itens, informa que está cheia
        if len(self.itens) == self.capacidade:
            print("Overflow! Pilha cheia.")
            return
        else:
            self.itens.append(item)

    # remove o item do top e retorna o item removido (pot)
    def desempilhar (self):
        # Se pilha = vazia, informa que não pode remover itens
        if self.vazia():
            print("Underflow! Não é possível remover: pilha vazia.")
            return None
        else:
            return self.itens.pop()


    # retorna o item do topo da pilha
    def topo(self): # top
        if not self.vazia():
            return self.itens[-1]

    # retorna o numero de elementos da pilha
    def tamanho(self):
        return len(self.itens)

    # esvaziar pilha, remove todos os dados
    def esvaziar(self):
        self.itens.clear()

    # imprime todos os itens existentes na pilha
    def imprimir (self):
```

```python
    # imprime todos os itens existentes na pilha
    def imprimir (self):
        print(f"\t",self.itens)
```

```python
# Testando a PILHA
entrada = [2, 4, 6, 8, 10]
pilha = Pilha(capacidade=5)
for i in entrada:
    pilha.empilhar(i)
    pilha.imprimir()


print("1-Item no Topo da pilha:", pilha.topo())
pilha.imprimir()
print("2-Tamanho da pilha:", pilha.tamanho())
pilha.imprimir()
print("3-Desempilhar (remove item do topo):", pilha.desempilhar())
pilha.imprimir()

item = 12
pilha.empilhar(item)
print(f'4-Empilhar (adiciona o item {item} no topo da pilha)')
pilha.imprimir()
print("5-Item no Topo da pilha:", pilha.topo())
pilha.imprimir()
print("6-Tamanho da pilha:", pilha.tamanho())
pilha.imprimir()

item = 14
pilha.empilhar(item)
print(f'7-Empilhar (adiciona o item {item} no topo da pilha)')
pilha.imprimir()
print("8-Item no Topo da pilha:", pilha.topo())
pilha.imprimir()
print("9-Tamanho da pilha:", pilha.tamanho())

pilha.esvaziar()
pilha.imprimir()
print("10-Tamanho da pilha:", pilha.tamanho())
pilha.imprimir()
pilha.desempilhar()
pilha.imprimir()
```

```
...         [2]
            [2, 4]
            [2, 4, 6]
            [2, 4, 6, 8]
            [2, 4, 6, 8, 10]
1-Item no Topo da pilha: 10
            [2, 4, 6, 8, 10]
2-Tamanho da pilha: 5
            [2, 4, 6, 8, 10]
3-Desempilhar (remove item do topo): 10
            [2, 4, 6, 8]
4-Empilhar (adiciona o item 12 no topo da pilha)
            [2, 4, 6, 8, 12]
5-Item no Topo da pilha: 12
            [2, 4, 6, 8, 12]
6-Tamanho da pilha: 5
            [2, 4, 6, 8, 12]
Overflow! Pilha cheia.
7-Empilhar (adiciona o item 14 no topo da pilha)
            [2, 4, 6, 8, 12]
8-Item no Topo da pilha: 12
            [2, 4, 6, 8, 12]
9-Tamanho da pilha: 5
            []
10-Tamanho da pilha: 0
            []
Underflow! Não é possível remover: pilha vazia.
            []
```

10. Questão

Neste exercício, construí uma fila com tamanho fixo, e controle de índices(início e fim).
Explicação (os detalhes linha a linha estão comentados no código):

- Criei uma classe no primeiro bloco de código com as seguintes funções:
  - **init** - Iniciar/Criar nova fila onde são definidos os ponteiros início e fim, além do tamanho inicial
  - **vazia** - verifica se fila está vazia
  - **enfileirar/push (adicionar)** - adiciona um novo item no final da fila
  - **desenfileirar** - remove o item do início da fila e retorna o item removido
  - **Início** - retorna o item no inicio da fila
  - **tamanho_fila** - retorna o número de elementos da fila
  - **esvaziar** - esvaziar fila, remove todos os dados
  - **imprimir** - imprime todos os itens existentes na fila, início, fim e tamanho atual

10 - Fila com Ponteiros

```python
class Fila:
    # cria uma nova fila vazia
    def __init__(self, capacidade):
        self.capacidade = capacidade        # adicionar atributo ao objeto
        self.itens = [None] * capacidade
        self.inicio = 0
        self.fim = 0
        self.tamanho = 0

    # verifica se a fila está vazia
    def vazia(self):
        return self.tamanho == 0

    # adiciona um novo item no final da fila (push)
    def enfileirar (self, item):
        # Se fila = capacidade máxima de itens, informa que está cheia
        if self.tamanho == self.capacidade:
            print("Overflow! Fila cheia.")
            return
        else:
            self.itens[self.fim] = item
            self.fim = (self.fim + 1) % self.capacidade
            self.tamanho += 1

    # remove o último da fila (pop)
    def desenfileirar (self):
        # Se fila = vazia, informa que não pode remover itens
        if self.tamanho == 0:
            print("Underflow! Não é possível remover: fila vazia.")
            return None
        else:
            item = self.itens[self.inicio]
            self.itens[self.inicio] = None
            self.inicio = (self.inicio + 1) % self.capacidade
            self.tamanho -= 1
            return item

    # retorna o primeiro item na fila
    def primeiro(self): # top
        if self.tamanho == 0:
            return None
        else:
            return self.itens[self.inicio]
```

```python
    # retorna o primeiro item na fila
    def primeiro(self): # top
        if self.tamanho == 0:
            return None
        else:
            return self.itens[self.inicio]

    # retorna o numero de elementos na fila
    def tamanho_fila(self):
        return self.tamanho

    # esvaziar fila, remove todos os dados
    def esvaziar(self):
        self.itens = [None] * self.capacidade
        self.inicio = 0
        self.fim = 0
        self.tamanho = 0

    # imprime todos os itens existentes na fila
    def imprimir(self):
        print(f"\t{self.itens} | inicio: {self.inicio} | fim: {self.fim} | tamanho: {self.tamanho}")
```

```python
# Testando a FILA
fila = Fila(capacidade=5)
fila.imprimir()
cont = 0

for i in range(4, 13, 2):
    cont += 1
    fila.enfileirar(i)
    print(f'{cont}-Enfileirar (adiciona o item {i})' )
    print(f"\tPrimeiro item:", fila.primeiro())
    fila.imprimir()

item_removido = fila.desenfileirar()
print(f"6-Desenfileirar (remove item {item_removido}):")
fila.imprimir()
print(f"7-Primeiro item:", fila.primeiro())
fila.imprimir()

item_removido = fila.desenfileirar()
print(f"8-Desenfileirar (remove item {item_removido}):")
fila.imprimir()
print(f"9-Primeiro item:", fila.primeiro())
fila.imprimir()
```

```python
item_removido = fila.desenfileirar()
print(f"6-Desenfileirar (remove item {item_removido}):")
fila.imprimir()
print(f"7-Primeiro item:", fila.primeiro())
fila.imprimir()

item_removido = fila.desenfileirar()
print(f"8-Desenfileirar (remove item {item_removido}):")
fila.imprimir()
print(f"9-Primeiro item:", fila.primeiro())
fila.imprimir()

item = 4
fila.enfileirar(item)
print(f'10-Enfileirar (adiciona o item {item})')
fila.imprimir()
print(f"11-Primeiro item:", fila.primeiro())
fila.imprimir()
```

```
        [None, None, None, None, None] | inicio: 0 | fim: 0 | tamanho: 0
1-Enfileirar (adiciona o item 4)
        Primeiro item: 4
        [4, None, None, None, None] | inicio: 0 | fim: 1 | tamanho: 1
2-Enfileirar (adiciona o item 6)
        Primeiro item: 4
        [4, 6, None, None, None] | inicio: 0 | fim: 2 | tamanho: 2
3-Enfileirar (adiciona o item 8)
        Primeiro item: 4
        [4, 6, 8, None, None] | inicio: 0 | fim: 3 | tamanho: 3
4-Enfileirar (adiciona o item 10)
        Primeiro item: 4
        [4, 6, 8, 10, None] | inicio: 0 | fim: 4 | tamanho: 4
5-Enfileirar (adiciona o item 12)
        Primeiro item: 4
        [4, 6, 8, 10, 12] | inicio: 0 | fim: 0 | tamanho: 5
6-Desenfileirar (remove item 4):
        [None, 6, 8, 10, 12] | inicio: 1 | fim: 0 | tamanho: 4
7-Primeiro item: 6
        [None, 6, 8, 10, 12] | inicio: 1 | fim: 0 | tamanho: 4
8-Desenfileirar (remove item 6):
        [None, None, 8, 10, 12] | inicio: 2 | fim: 0 | tamanho: 3
9-Primeiro item: 8
        [None, None, 8, 10, 12] | inicio: 2 | fim: 0 | tamanho: 3
10-Enfileirar (adiciona o item 4)
        [4, None, 8, 10, 12] | inicio: 2 | fim: 1 | tamanho: 4
11-Primeiro item: 8
        [4, None, 8, 10, 12] | inicio: 2 | fim: 1 | tamanho: 4
```

11. QUESTÃO

Neste exercício, construí uma classe **ReverterString** onde estão as principais funções:
Explicação (os detalhes linha a linha estão comentados no código):

- Criei uma classe no primeiro bloco de código com as seguintes funções:
  - **init** - Iniciar/Criar nova pilha
  - **vazia** - verifica se pilha está vazia
  - **empilhar/push (adicionar)** - adiciona um novo item no topo da pilha
  - **desempilhar** - remove o item do top e retorna o item removido
  - **topo** - retorna o item do topo da pilha
  - **tamanho** - retorna o número de elementos da pilha
  - **esvaziar** - esvaziar pilha, remove todos os dados
  - **imprimir** - imprime todos os itens existentes na pilha
- Depois criei uma segundo bloco de código código para demonstrar o que foi pedido:
  - Inicialmente faço a criação de um lista com 5 listas de strings dentro
  - Depois percorro um loop “for” na lista acima e a cada string empilho e depois eu desempilho a string e então escrevo em tela a string empilhada/original e depois ela invertida(desempilhada)

11 - Pilha reversão de String

```python
class ReverterString:
    # cria uma nova pilha vazia com capacidade máxima
    def __init__(self, capacidade):
        self.capacidade = capacidade        # adicionar atributo ao objeto
        self.itens = []

    # verifica se a pilha está vazia
    def vazia(self):
        return not self.itens

    # adiciona um novo item no topo da pilha (push)
    def empilhar (self, item):
        # Se pilha = capacidade máxima de itens, informa que está cheia
        if len(self.itens) == self.capacidade:
            print("Overflow! Pilha cheia.")
            return
        else:
            self.itens.append(item)

    # remove o item do top e retorna o item removido (pot)
    def desempilhar (self):
        # Se pilha = vazia, informa que não pode remover itens
        if self.vazia():
            print("Underflow! Não é possível remover: pilha vazia.")
            return None
        else:
            return self.itens.pop()

    # imprime todos os itens existentes na pilha
    def imprimir_apos_empilhar (self):
        resultado = "".join(self.itens)
        print(f"String após empilhar: {resultado}")
```

```python
# Testando a PILHA - com diferentes strings
listas = [
    ["ABCDEF"],
    ["Pythonic"],
    ["StackClass"],
    ["DataStruct12"],
    ["ReverseStackABC"]
]

for item in listas:
    item = item[0]  #trasforma em string
    pilha = ReverterString(capacidade=len(item))

    print(f"\nOriginal: {pilha.itens}")
    for i, letra in enumerate(item):
        pilha.empilhar(letra)

    pilha.imprimir_apos_empilhar()

    string_invertida = ""
    while not pilha.vazia():
        letra_removida = pilha.desempilhar()
        string_invertida += letra_removida

    print(f"String invertida: {string_invertida}")
```

```
Original: []
String após empilhar: ABCDEF
String invertida: FEDCBA

Original: []
String após empilhar: Pythonic
String invertida: cinohtyP

Original: []
String após empilhar: StackClass
String invertida: ssalCkcatS

Original: []
String após empilhar: DataStruct12
String invertida: 21tcurtSataD

Original: []
String após empilhar: ReverseStackABC
String invertida: CBAkcatSesreveR
```